MiniTCP Protocol Design

Computer Networks Assignment 2 — Protocol Documentation

Download Manager Project

1. Overview

MiniTCP is a simple, reliable, connection-oriented transport protocol implemented entirely on top of **UDP**. It is the foundation for the Download Manager application (Part 2), which uses MiniTCP to transfer file chunks between a client and a server.

MiniTCP provides the four capabilities required by the assignment:

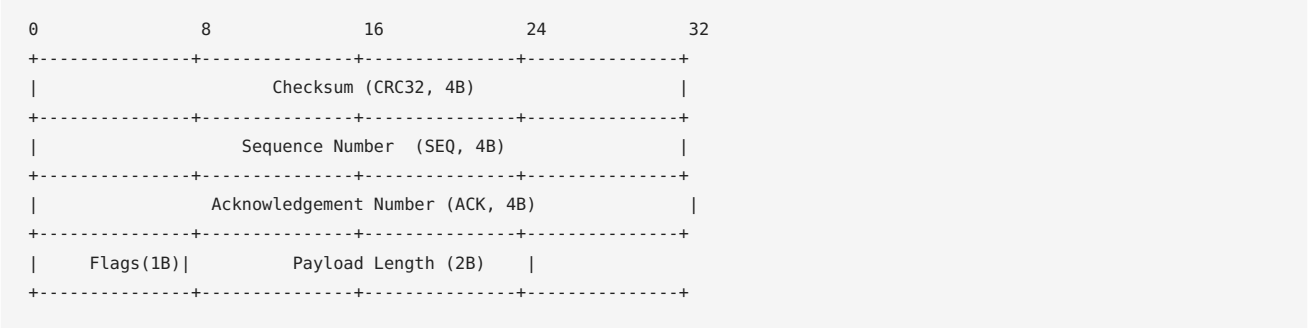
1. Three-way connection establishment (handshake)
2. Sequence numbers on data packets
3. Acknowledgement (ACK) of received packets
4. Timeout-based loss detection with retransmission
5. Explicit connection termination

It is implemented in Python in the `minitcp` package:

- `minitcp/protocol.py` — packet format, encoding/decoding, checksum
- `minitcp/connection.py` — handshake, reliable send/recv, retransmission, termination

2. Packet Format

Every MiniTCP packet is a single UDP datagram with a 15-byte header followed by up to 1024 bytes of payload:



```
|                       Payload (0-1024 B)                    |
+-------------------------------------------------------------+
```

Field	Size	Meaning
Checksum	4 bytes	CRC32 of (header without checksum + payload); used to detect corruption
SEQ	4 bytes	Sequence number of *this* packet
ACK	4 bytes	Number being acknowledged (meaningful when ACK flag set)
Flags	1 byte	Bitmask, see below
Length	2 bytes	Length of the payload in bytes
Payload	0–1024B	Application data

2.1 Flags

Flag	Value	Meaning
SYN	0x01	Connection request (handshake)
ACK	0x02	Acknowledges the packet numbered in the ACK field
FIN	0x04	Terminates a connection or ends a data stream
DATA	0x08	Payload carries application data
REQ	0x10	Application-level request (e.g. “give me chunk N”)
ERR	0x20	Error notification

Flags may be combined, e.g. `SYN | ACK` in the second handshake step.

3. Connection Establishment (3-Way Handshake)

Because UDP has no built-in notion of a “connection”, MiniTCP creates one explicitly, and — to allow the server to service many **independent, concurrent** connections (a hard requirement of the Download Manager, which opens 9 parallel connections) — it borrows a technique from TFTP: every new connection gets its **own ephemeral UDP socket** on the server side.

```
Client                                  Server (well-known port P)
  |                                            |
  |------------------ SYN(seq=x) --------------->|
  |                                            |  spawns a worker
  |                                            |  thread bound to
  |                                            |  a NEW ephemeral
  |                                            |  port E
  |<------------- SYN|ACK(seq=y, ack=x+1) -------|  (sent FROM port E)
  |                                            |
  |------------------ ACK(ack=y+1) -------------->| (sent TO port E)
  |                                            |
  |     == connection established on port E ==   |
```

1. **Client → Server**: sends `SYN` with an initial random sequence number `x` to the server’s well-known port.
2. **Server → Client**: the listener thread hands the SYN to a new worker thread, which binds a *fresh* ephemeral UDP socket and replies with `SYN|ACK` (`seq=y` , `ack=x+1`) from that new port.
3. **Client → Server**: client now knows the server’s per-connection port (source of the SYN-ACK datagram) and confirms with `ACK` (`ack=y+1`) sent to that port.

All further datagrams for this logical connection flow only between the client's ephemeral port and the server's newly-created ephemeral port — completely isolated from every other connection. Each handshake step is retried (default: 10 attempts, 1 s timeout) in case a SYN or SYN-ACK is lost.

4. Data Transfer: Sequencing and Acknowledgement

Data is transferred with a **stop-and-wait ARQ** scheme:

- The sender splits the payload into chunks of at most 1024 bytes.
- Each chunk is sent as a `DATA` packet carrying an incrementing sequence number.
- The sender blocks until it receives an `ACK` for that exact sequence number before sending the next chunk.
- The receiver accepts a `DATA` packet only if its sequence number matches the next *expected* sequence number, then immediately replies with `ACK`. Duplicate (already-seen) packets are re-ACKed without being re-delivered to the application, so a lost ACK never causes duplicated data.
- A CRC32 checksum lets the receiver silently discard corrupted datagrams — from the sender's point of view this looks exactly like a lost packet, and is handled by the same timeout logic.

Stop-and-wait was chosen over a sliding window for clarity and robustness within the assignment's scope; the required parallelism is instead achieved at the application layer, since the Download Manager already runs 9 MiniTCP connections concurrently, each on its own thread.

5. Timeout and Retransmission

Every packet that expects a reply (`SYN`, `DATA`, `FIN`, `REQ`) is sent through the same primitive:

```
send packet
wait for matching ACK, up to `timeout` seconds
if no (matching) ACK arrives:
    increase timeout (exponential backoff, capped at 3s)
    retransmit the SAME packet
    (repeat up to MAX_RETRIES = 12 times)
raise an error if still unacknowledged after MAX_RETRIES
```

Starting timeout is 0.4 s. This mechanism is what makes the protocol *reliable*: even under significant packet loss, data eventually gets through, or the connection fails cleanly and predictably instead of hanging or silently corrupting the transfer.

The implementation includes an optional artificial packet-loss mode (`--loss <probability>` on both `client.py` and `server.py`) used to demonstrate and test this mechanism — see Section 7.

6. Connection Termination

The end of a data stream is signalled with a `FIN` packet (itself sent through the reliable send-and-wait-for-ACK primitive described above), which the receiver ACKs before returning the reassembled data to the caller. For connections that need a full duplex close (no more data pending on either side), the protocol also supports a symmetric 4-way close:

```
Side A                    Side B
  |---------- FIN --------->|
  |<---------- ACK ----------|
  |<---------- FIN ----------|
  |---------- ACK --------->|
```

exposed as `close_active()` / `close_passive()` in `minitcp/connection.py`.

7. Reliability Demonstration

To verify the retransmission logic under real loss (not just in theory), both `server.py` and `client.py` accept a `--loss` parameter that randomly drops a fraction of *outgoing* packets before they are sent, simulating an unreliable network.

Test performed: a 1.5 MB file was downloaded through 9 parallel MiniTCP connections with **10% independent packet loss injected on both the client and the server**. The transfer completed successfully and the SHA-256 checksum of the downloaded file matched the original exactly. Server- and client-side logs (`-v` flag) showed dozens of “timeout / retransmitting” events being triggered and resolved automatically, confirming that lost `SYN`, `DATA`, `ACK` and `FIN` packets are correctly detected and retransmitted.

8. Design Summary

Requirement	Where implemented
3-way handshake	`connection.connect()` / `_accept_worker()`
Sequence numbers	`seq` field on every packet, tracked per connection
ACK of received packets	`ACK` flag, `MiniTCPConnection.recv()` / `.send()`
Timeout-based loss detection	`_send_and_wait_ack()`
Retransmission	same function, up to `MAX_RETRIES` with backoff
Connection termination	`FIN` / `ACK` at end of `send()`; `close_active/passive`